SGD Optimizer & Learning Rate Scheduling

1. Why Optimization Matters

为什么优化器很关键?

深度学习训练的本质是一个 高维非凸优化问题：
• 参数维度：几万到几亿
• 损失曲面：多鞍点、多扭曲谷地
• 梯度变化剧烈、方向不稳
• 学习率敏感、容易发散

优化器决定模型能否真的“学到东西”。
SGD 是所有优化器的基石。

2. Gradient Descent → SGD：从确定性到随机性

2.1 Empirical Risk Minimization (ERM)

给定数据集：

$$\mathcal{D} = (x_i, y_i)_{i=1}^{N}$$

经验风险：

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i; \theta), y_i)$$

2.2 Full Gradient Descent（全量梯度下降）

每次更新:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

缺点：
• 每次计算梯度需要遍历全数据（慢）
• 对曲面几何敏感，容易陷入“弯曲谷地”导致训练卡住
• 不适用于大数据集（ImageNet 级别就完全不可行）

→ 所以出现了 SGD。

2.3 Stochastic Gradient Descent（随机梯度下降）

使用 mini-batch 代替整体数据：

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \ell(x_b, y_b; \theta_t)$$

其中 mini-batch 通常为 32、64、128或者更多。

SGD 的重要特性：

1. 噪声高（variance 高）
   梯度方向不再稳定，但这种“随机抖动”有一个奇妙现象：

SGD 更容易逃出鞍点（saddle point），而 GD 会卡住。

2. 计算快（Fast iteration）
   每次更新不需要跑完整数据。
3. 泛化好
   SGD 的噪声起到类似“正则化”的效果，使模型不容易过拟合。

2.4 Why Noise Helps?（非常重要的工程直觉）

深度神经网络的损失曲面具有大量“鞍点”（saddle point），而不是“局部最小值”。

GD 的问题：
• 在鞍点附近梯度非常小
• 更新停止，模型不再进步

SGD 的噪声能让你抖出鞍点，继续下降。

图像直觉
• 想象一个马鞍型表面
• 全量梯度下降在顶部徘徊
• SGD 的噪声把你“抖下来”

3. Weight Decay

3.1 L2 Regularization（数学定义）

$$L_{\text{total}} = L(\theta) + \lambda|\theta|^2$$

梯度为：

$$\nabla L_{\text{total}} = \nabla L + \lambda\theta$$

3.2 SGD 更新时的 Weight Decay（耦合式）

$$\theta_{t+1} = \theta_t - \eta(\nabla L + \lambda\theta_t)$$

几何意义：

它让更新速度与参数大小有关：参数越大，衰减越大。

等价于：
• 你在不断“推模型往 0 靠拢”
• 模型被迫使用更小的权重（更平滑、更泛化）

4. Gradient Explosion & Gradient Clipping（梯度爆炸与梯度剪裁）

4.1 What causes exploding gradients?

• Recurrent connections（如 RNN）反复乘 Jacobian，指数放大
• 深模型链式法则导致乘积疯狂增长
• 学习率太大
• 权值初始化不当

4.2 Clip-by-Value

$$g \leftarrow \text{clamp}(g, -c, c)$$

简单粗暴，但不常用。

4.3 Clip-by-Norm（全局梯度裁剪）

计算所有梯度的范数：

$$|g| = \sqrt{\sum_i |g_i|^2}$$

如果超过阈值：

$$g_i \leftarrow g_i \cdot \frac{c}{|g|}$$

这是 PyTorch 和深度学习训练中最常用的剪裁方式。

工程直觉：
• 不是逐个梯度“截断”，而是保持大方向不变
• 只是缩小“步幅”，保持优化稳定

5. PyTorch Optimizer Design（理解 Optimizer 的内部结构）

一个优化器要具备三点：

1. 保存需要更新的参数（param_groups）
2. 提供 step() 更新参数
3. 提供 zero_grad() 清空梯度

PyTorch 只是把这些封装得很优雅，本质仍然很简单。

6. MySGD

class MySGD:
def **init**(self, params, lr=0.01, weight_decay=0.0, max_grad_norm=None):
self.params = list(params)
self.lr = lr
self.weight_decay = weight_decay
self.max_grad_norm = max_grad_norm

```
def zero_grad(self):
    for p in self.params:
        if p.grad is not None:
            p.grad.zero_()

def _clip_grad_norm(self):
    if self.max_grad_norm is None:
        return
    # Compute total grad norm
    total_norm = torch.sqrt(sum((p.grad.norm() ** 2 for p in self.params))
    if total_norm > self.max_grad_norm:
        scale = self.max_grad_norm / (total_norm + 1e-6)
        for p in self.params:
            p.grad.mul_(scale)

def step(self):
    # 1. Clip gradients
    self._clip_grad_norm()

    # 2. Update parameters
    for p in self.params:
        if p.grad is None:
            continue

        grad = p.grad

        # Weight decay (coupled)
        if self.weight_decay != 0:
            grad = grad + self.weight_decay * p.data

        # SGD update
        p.data = p.data - self.lr * grad
```

6. Learning Rate Scheduling（为什么我们需要 LR 调度？）

训练的两大阶段：

1）前期：探索（exploration）

需要较大的学习率：
• 更新幅度大
• 跳出糟糕区域
• 快速下降

2）后期：收敛（convergence）

需要较小的学习率：
• 精细更新
• 避免震荡

因此学习率无法固定，需要“动态调整”。

7 常见学习率策略的公式与优缺点

(1) Step Decay

公式：

$$\mathrm{lr}_t = \mathrm{lr}_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

优点：
• 简单
缺点：
• 学习率突然跳变可能造成训练曲线震荡

(2) Exponential Decay

$$\mathrm{lr}_t = lr_0 \cdot \gamma^t$$

优点：平滑衰减
缺点：后期衰减过快

(3) Cosine Annealing

$$lr_t = \frac{1}{2} lr_0 \left(1 + \cos \frac{t\pi}{T}\right)$$

优点：
• 平滑下降
• 实践中常优于 StepLR

缺点：

• 训练中期下降可能过快

(4) Warmup

$$lr_t = lr_{\max} \cdot \frac{t}{T_{\text{warm}}}$$

优点：

• 避免初始梯度爆炸

8. Implementing MyStepLR

class MyStepLR:

def **init**(self, optimizer, step_size, gamma=0.1):

self.optimizer = optimizer

self.step_size = step_size

self.gamma = gamma

self.last_epoch = 0

```
def step(self):
    self.last_epoch += 1
    if self.last_epoch % self.step_size == 0:
        self.optimizer.lr *= self.gamma
```

非常直观：每 step_size 个 epoch 衰减一次 lr。